

UNED IA 2026/27

FUNDAMENTOS DE PROGRAMACIÓN

Código 71901020

HECHO PARA EMPEZAR SIN CONOCIMIENTOS PREVIOS

No tienes que memorizar símbolos a ciegas. Primero entenderás qué problema resuelve cada idea, después verás un dibujo mental, luego seguiremos el código línea por línea y finalmente practicarás.

Incluye	Qué encontrarás
15 temas oficiales	Explicados en el mismo orden de la guía pública 2026/27.
Ejemplos	Código pequeño, tabla de seguimiento y comprobación del resultado.
Práctica	Microejercicios, preguntas tipo test y problemas de programación.
Orientación al examen	Patrones observados en el archivo reunido: 42 enunciados y 16 soluciones oficiales.
Colores constantes	Azul = idea, verde = regla segura, naranja = cuidado, rojo = error frecuente, morado = memoria.

Manual completo desde cero - versión 1 - septiembre de 2026

Antes de estudiar: entiende el terreno

QUÉ DICE LA GUÍA OFICIAL 2026/27

La asignatura es de **primer curso y primer cuatrimestre**, tiene **6 ECTS** y estima una dedicación total de **150 horas**. No exige conocimientos previos de programación.

La prueba presencial indicada es **mixta**: 10 preguntas tipo test y 1 ejercicio de programación, con 120 minutos y sin material auxiliar. El test aporta el 30% y el ejercicio el 60%; el examen aporta el 90% de la nota final.

El test es eliminatorio: la guía señala 8 aciertos, o 6 si se han realizado y aprobado las prácticas. Las PEC son voluntarias, son aproximadamente cuatro y deben completarse correctamente para ser consideradas. El algoritmo definitivo de calificación se publica en la web de la asignatura.

Qué significa esto para ti

- No basta con saber programar un ejercicio largo: hay que dominar también definiciones y pequeños fragmentos de código para superar el test.
- No basta con memorizar definiciones: el ejercicio grande suele exigir diseñar un Tipo Abstracto de Datos con registros, arrays y operaciones.
- Las prácticas pueden reducir el umbral del test y, sobre todo, convierten las ideas en una habilidad real. Conviene hacerlas cuando el curso publique sus instrucciones.

REGLA DE HONESTIDAD

El archivo histórico permite ver patrones, no adivinar el examen. Este manual prioriza los contenidos repetidos, pero cubre los 15 temas oficiales. La evaluación y las PEC deben verificarse otra vez en el curso virtual cuando esté abierto.

Mapa de prioridades observado en los exámenes

Prioridad	Bloques	Por qué
MUY ALTA	Seguir código; control; funciones y parámetros; arrays y registros; diseño de TAD	Aparecen de forma persistente en test y el TAD domina el ejercicio largo reciente.
ALTA	Tipos, BNF, metodología, complejidad, recorridos, búsqueda y ordenación	Producen preguntas conceptuales y son herramientas necesarias para el ejercicio.
MEDIA	Punteros, memoria dinámica, excepciones, recursión, módulos	No aparecen en todos los modelos, pero pueden decidir varias preguntas del test.

Cómo estudiar sin memorizar a ciegas

Una sesión de 60 minutos

Minutos	Trabajo	Objetivo
0-10	Lee vocabulario e imagen mental	Saber de qué estamos hablando.
10-30	Sigue un ejemplo con lápiz	Predecir cada cambio antes de verlo.
30-50	Resuelve ejercicios sin mirar	Recuperar la idea de memoria.
50-60	Corrige y explica en voz alta	Convertir el error en una regla propia.

CÓDIGO DE COLORES PARA MEMORIZAR

AZUL: concepto que debes comprender.

VERDE: regla fiable o comprobación.

NARANJA: decisión o paso que exige atención.

ROJO: error típico de examen.

MORADO: frase corta para recordar.

El único dibujo que necesitas al principio

Casi todo lo que harás en programación cabe en este recorrido. Un problema del mundo real entra por la izquierda y un resultado comprobable sale por la derecha.

1. Problema	2. Datos	3. Algoritmo	4. Programa	5. Resultado
Qué se pide	Qué entra y qué guardo	Pasos exactos	Pasos escritos en C+/-	Salida y pruebas

Las cinco preguntas universales

- ¿Qué datos recibo?
- ¿Qué resultado debo producir?
- ¿Qué variables necesito para recordar información?
- ¿Qué decisiones y repeticiones hacen falta?
- ¿Cómo compruebo que funciona en casos normales y casos límite?

FRASE PARA MEMORIZAR

ENTRA - GUARDO - DECIDO - REPITO - DEVUELVO - COMPRUEBO. Si un código te asusta, vuelve a esas seis palabras.

TEMA 1

Introducción: de un problema a un programa

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo transformamos una necesidad expresada con palabras en pasos que un ordenador puede ejecutar sin adivinar nada?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Algoritmo	Secuencia finita y precisa de pasos que resuelve un problema.
Programa	Algoritmo escrito en un lenguaje de programación.
Código fuente	Texto que escribe la persona programadora.
Compilador	Herramienta que traduce el código y detecta errores de forma.
Sintaxis	Reglas que dicen cómo se escribe una instrucción válida.
Semántica	Significado de una instrucción válida.
BNF	Notación para describir la sintaxis de un lenguaje.

La idea sin tecnicismos

Un ordenador es muy rápido, pero no interpreta intenciones. Si decimos 'ordena estos números', falta explicar dónde están, qué significa ordenar y qué hacer con el resultado. Un algoritmo elimina esa ambigüedad.

IMAGEN MENTAL

Piensa en una receta. Los ingredientes son los datos, los pasos son el algoritmo y el plato terminado es el resultado. Si un paso dice 'cocina un poco', una persona puede improvisar; un ordenador no.

Del código fuente al resultado

Etapas	Qué ocurre	Error típico
Editar	Escribes instrucciones.	Confundir una letra o un símbolo.
Compilar	Se comprueba la sintaxis y se traduce.	Falta un punto y coma o una llave.
Ejecutar	El ordenador realiza las instrucciones.	El programa termina, pero calcula mal.
Probar	Comparamos lo obtenido con lo esperado.	Probar solo un caso fácil.

BNF, explicada con una frase

BNF describe **qué formas son válidas**, no qué resultado producen. En los exámenes recientes aparece esta diferencia: BNF especifica la sintaxis, no la semántica ni la complejidad.

```
<dígito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9  
<entero> ::= <dígito> { <dígito> }
```

Se lee así: un entero contiene un dígito y puede repetir más dígitos. Los metasímbolos habituales del curso son: **{ }** para repetición, **[]** para opción y **()** para agrupación. Comprueba siempre la convención exacta usada en el material del curso.

Ejemplo resuelto: calcular el precio final

Problema: conocemos un precio de 80 euros y un descuento del 15%. Queremos el importe que se paga.

Paso	Pensamiento	Resultado parcial
1	Datos de entrada	precio = 80; descuento = 15
2	Calcular lo que se descuenta	$80 \times 15 / 100 = 12$
3	Restarlo	$80 - 12 = 68$
4	Comprobar	68 es menor que 80 y la diferencia es 15%.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. ¿Un programa que compila necesariamente da el resultado correcto?
2. ¿BNF describe el significado o la forma de las instrucciones?
3. Ordena estas fases: probar, ejecutar, editar, compilar.

SOLUCIONES EXPLICADAS

1. No. Compilar indica que la forma es aceptable. Todavía puede existir un error lógico.
2. La forma, es decir, la sintaxis.
3. Editar, compilar, ejecutar y probar.

TEMA 2

Elementos básicos de programación

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cuáles son las piezas mínimas con las que se escribe una instrucción y cómo se evalúa una expresión?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Identificador	Nombre de una variable, función o tipo.
Literal	Valor escrito directamente: 7, 3.5 o 'A'.
Operador	Símbolo que realiza una operación: +, -, *, /, %, <, ==.
Expresión	Combinación que produce un valor.
Sentencia	Instrucción completa que realiza una acción.
Bloque	Grupo de sentencias encerrado entre llaves.
Palabra reservada	Palabra del lenguaje que no puede usarse como nombre, como int o while.

Desmontemos una línea

```
int total = precio * cantidad;
```

Parte	Qué es	Qué hace
int	tipo	Indica que total guardará un número entero.
total	identificador	Es el nombre elegido para la variable.
=	asignación	Guarda el resultado de la derecha en la izquierda.
precio * cantidad	expresión	Multiplica dos valores y produce uno nuevo.
;	fin de sentencia	Marca dónde termina la instrucción.

CUIDADO

En C+/- se distinguen mayúsculas y minúsculas. **diaSemana** y **DIASEMANA** son nombres diferentes. Además, **=** guarda un valor y **==** compara dos valores.

Aritmética que suele engañar

Expresión	Resultado	Por qué
7 / 2	3	Si ambos operandos son enteros, se descarta la parte decimal.
7.0 / 2	3.5	Hay un valor real, por lo que la división es real.
7 % 2	1	% devuelve el resto de la división entera.
2 + 3 * 4	14	La multiplicación se evalúa antes que la suma.
(2 + 3) * 4	20	Los paréntesis cambian el orden.

Ejemplo resuelto: seguir una expresión

```
int a = 11;
int b = 4;
int c = a / b + a % b;
```

Primero, $11 / 4$ vale 2. Después, $11 \% 4$ vale 3. Finalmente, $c = 2 + 3 = 5$. No aparece 2,75 porque la división es entera.

Expresiones lógicas

Operador	Lectura	Ejemplo
!	NO	!(edad >= 18) es cierto si edad es menor que 18.
&&	Y	llueve && haceFrio exige que ambas sean ciertas.
	O	esSabado esDomingo exige al menos una cierta.
<, <=, >, >=	comparación	nota >= 5
==, !=	igual, distinto	opcion != 0

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. Calcula $17 / 5$ y $17 \% 5$ usando enteros.
2. ¿Qué diferencia hay entre $x = 3$ y $x == 3$?
3. ¿Son el mismo identificador total y Total?

SOLUCIONES EXPLICADAS

1. $17 / 5$ es 3 y $17 \% 5$ es 2: $17 = 3 \times 5 + 2$.
2. $x = 3$ guarda 3 en x. $x == 3$ pregunta si x vale 3.
3. No. El lenguaje distingue mayúsculas y minúsculas.

TEMA 3

Constantes y variables

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo reserva el programa espacios para recordar datos y qué reglas limitan lo que puede guardarse en cada espacio?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Variable	Casilla con nombre cuyo contenido puede cambiar.
Constante	Nombre asociado a un valor que no debe cambiar.
Tipo	Conjunto de valores permitidos y operaciones válidas.
Declarar	Indicar al programa el nombre y el tipo de una variable.
Inicializar	Dar el primer valor al declarar.
Ámbito	Zona del programa donde un nombre es visible.
Conversión	Transformación de un valor de un tipo a otro.

La imagen de las cajas etiquetadas

Imagina una estantería. Cada variable es una caja con una etiqueta. El tipo decide qué cabe en la caja. Una caja `int` guarda enteros; una `float`, números con decimales; una `char`, un carácter; una `bool`, verdadero o falso.

```
int edad = 52;
float temperatura = 18.5;
char inicial = 'C';
bool terminado = false;
const int MAXIMO = 100;
```

REGLA SEGURA

Declara la variable lo más cerca posible de donde se usa, inicialízala antes de leerla y elige un nombre que explique su función.

Seguimiento: una variable cambia, no viaja en el tiempo

```
int x = 4;
int y = x + 3;
x = 10;
```

Después de	x	y	Explicación
<code>int x = 4</code>	4	-	Se crea x con 4.
<code>int y = x + 3</code>	4	7	Se usa el valor actual de x.
<code>x = 10</code>	10	7	Cambiar x no recalcula y.

Conversión de tipos

Una conversión explícita dice con claridad qué tipo queremos. Por ejemplo, `int(3.9)` produce 3: no redondea, elimina la parte decimal. Convertir puede perder información, por eso debe hacerse de forma

consciente.

Ámbito y ocultación accidental

```
int valor = 8;           // global

int main() {
    int valor = 3;       // local: oculta al global dentro de main
    printf("%d", valor); // imprime 3
}
```

Cuando un nombre local tapa a otro nombre exterior se habla de **shadowing** u ocultación de nombres. No es el mismo concepto que ocultar la implementación de un TAD, que veremos más adelante.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. Tras `int a = 5; int b = a; a = 9;` ¿cuánto vale `b`?
2. ¿Qué tipo usarías para el número de habitaciones de una vivienda?
3. ¿Qué produce `int(6.8)`?

SOLUCIONES EXPLICADAS

1. `b` vale 5. La asignación copia el valor que `a` tenía en ese momento.
2. Un entero, porque el número de habitaciones se cuenta sin decimales.
3. Produce 6: se descarta la parte decimal.

TEMA 4

Metodología de desarrollo (I): especificar antes de programar

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo evitamos empezar a escribir código sin haber entendido exactamente qué debe hacer?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Especificación	Descripción precisa del problema y del resultado esperado.
Entrada	Dato que recibe el algoritmo.
Salida	Resultado que produce.
Precondición	Lo que debe cumplirse antes de ejecutar.
Postcondición	Lo que quedará cumplido al terminar.
Caso límite	Situación válida situada en un extremo: cero, vacío, máximo.
Prueba	Ejecución controlada con resultado esperado conocido.

El error más caro aparece antes del código

Si interpretamos mal el problema, podemos construir perfectamente el programa equivocado. Por eso separamos el **qué** del **cómo**. La especificación dice qué debe cumplirse; el algoritmo explica cómo conseguirlo.

Ejemplo: calcular una media válida

Elemento	Contenido
Entrada	Tres notas reales n_1 , n_2 y n_3 .
Precondición	Cada nota está entre 0 y 10.
Salida	La media aritmética.
Postcondición	$\text{media} = (n_1 + n_2 + n_3) / 3$ y también estará entre 0 y 10.
Casos	0,0,0; 10,10,10; valores diferentes; decimales.

```
float mediaTres(float n1, float n2, float n3) {  
    return (n1 + n2 + n3) / 3.0;  
}
```

COMPROBACIÓN VERDE

Con 4, 7 y 10 obtenemos 7. El resultado está entre el menor y el mayor dato, por lo que es razonable. Una comprobación de sentido común no demuestra todo, pero descubre muchos errores.

Plantilla para cualquier problema

1. Reescribe el objetivo con tus palabras.
2. Lista entradas, salidas y límites.
3. Crea al menos un ejemplo a mano.

4. Divide la solución en acciones pequeñas.
5. Programa una acción cada vez.
6. Prueba casos normales, límites y erróneos.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. Para dividir a / b , ¿qué precondition imprescindible falta?
2. Si buscas el máximo de una lista no vacía, ¿qué caso límite debes probar?
3. ¿La especificación explica el qué o el cómo?

SOLUCIONES EXPLICADAS

1. b debe ser distinto de cero.
2. Una lista de un solo elemento; ese elemento debe ser el máximo.
3. Explica el qué. El algoritmo explica el cómo.

TEMA 5

Estructuras básicas de la programación imperativa

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo expresamos que unas instrucciones van en orden, que elegimos un camino o que repetimos un trabajo?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Secuencia	Instrucciones ejecutadas una detrás de otra.
Selección	Elección entre alternativas según una condición.
Iteración	Repetición de acciones; también se llama bucle.
Condición	Expresión cuyo resultado es verdadero o falso.
Contador	Variable que cuenta cuántas veces ocurre algo.
Acumulador	Variable que va reuniendo un resultado, como una suma.
Traza	Tabla que anota los valores después de cada paso.

Las tres formas bastan para construir cualquier algoritmo

Estructura	Pregunta mental	Forma típica
Secuencia	¿Qué ocurre primero y después?	sentencia 1; sentencia 2;
Selección	¿Qué alternativa corresponde?	if (...) { ... } else { ... }
Iteración	¿Qué se repite y cuándo termina?	while (...), do-while o for

Ejemplo paso a paso: sumar del 1 al 4

```
int suma = 0;
int i = 1;
while (i <= 4) {
    suma = suma + i;
    i = i + 1;
}
```

Vuelta	i al entrar	suma antes	suma después	i después
1	1	0	1	2
2	2	1	3	3
3	3	3	6	4
4	4	6	10	5

Al llegar i a 5, la condición $5 \leq 4$ es falsa y el bucle termina. El resultado es 10.

while, do-while y for

Estructura	Úsala cuando	Mínimo de ejecuciones
while	Puede que no haya que entrar ninguna vez.	0
do-while	El cuerpo debe ejecutarse al menos una vez.	1
for	Hay una inicialización, condición y avance claros.	0

ERROR ROJO

Un bucle termina solo si algo acerca la condición a falso. Olvidar actualizar el contador suele producir un bucle infinito.

Ejemplo: selección

```
if (nota >= 5) {
    printf("Aprobado");
} else {
    printf("Pendiente");
}
```

Primero se evalúa `nota >= 5`. Solo se ejecuta uno de los dos bloques. En una selección debemos identificar **las alternativas** y **las condiciones para elegir cada alternativa**.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. ¿Cuántas veces como mínimo se ejecuta el cuerpo de do-while?
2. En el bucle de la suma, ¿qué ocurre si eliminamos `i = i + 1`?
3. ¿Un bucle es secuencia, selección o iteración?

SOLUCIONES EXPLICADAS

1. Una vez, porque la condición se comprueba después del cuerpo.
2. `i` siempre vale 1, la condición nunca se vuelve falsa y el bucle no termina.
3. Iteración: repite acciones mientras se cumple un criterio.

TEMA 6

Metodología de desarrollo (II): corrección y eficiencia

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo razonamos que un algoritmo termina, produce el resultado correcto y no realiza trabajo innecesario?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Refinamiento	Sustituir una tarea grande por pasos cada vez más concretos.
Corrección	El algoritmo cumple su especificación para las entradas válidas.
Invariante	Propiedad que sigue siendo cierta en cada vuelta de un bucle.
Función variante	Cantidad acotada que avanza estrictamente hacia el final.
Complejidad	Cómo crece el trabajo o la memoria al crecer la entrada.
n	Tamaño de la entrada: número de datos, elementos o pasos relevantes.
O grande	Forma de describir el crecimiento dominante para tamaños grandes.

Terminar no es cuestión de fe

Para justificar que un bucle termina buscamos una cantidad que cambie estrictamente y esté acotada. Si i empieza en 1, aumenta de uno en uno y el bucle exige $i \leq n$, la distancia $n - i$ disminuye hasta que la condición falla.

FRASE DE EXAMEN

Para garantizar la terminación se identifica una función estrictamente monótona y acotada. Monótona significa que siempre avanza en una dirección; acotada significa que no puede avanzar indefinidamente.

Big O sin miedo

No mide segundos exactos. Compara cómo crece el trabajo cuando n se hace grande. Si duplicar n duplica aproximadamente los pasos, hablamos de $O(n)$. Si cada elemento se combina con todos, suele aparecer $O(n^2)$.

Orden	Imagen mental	Ejemplo
$O(1)$	Abrir un cajón concreto.	Leer vector[5].
$O(\log n)$	Descartar la mitad cada vez.	Búsqueda binaria.
$O(n)$	Mirar todos una vez.	Sumar un vector.
$O(n^2)$	Cada uno con cada uno.	Dos bucles completos anidados.
$O(2^n)$	Las posibilidades se duplican.	Explorar todos los subconjuntos.

Orden habitual de menor a mayor crecimiento: $O(1) < O(\log n) < O(n) < O(n^2) < O(n^3) < O(2^n)$. Una complejidad polinómica puede ser n^2 , n^3 u otra potencia; sin conocer el grado no siempre puede compararse exactamente con una cuadrática.

Ejemplo: contar pasos

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        contador++;  
    }  
}
```

El bucle exterior hace n vueltas. Por cada vuelta, el interior hace n . En total: $n \times n = n^2$ incrementos. El orden es $O(n^2)$.

Invariante en el ejemplo de la suma

Antes de cada vuelta, suma contiene la suma desde 1 hasta $i-1$. Al añadir i y aumentarlo, la propiedad vuelve a ser cierta para la siguiente vuelta. Cuando termina, i es 5 y suma contiene $1+2+3+4$.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. Un bucle recorre una vez un vector de n elementos. ¿Qué orden tiene?
2. Dos bucles anidados recorren n elementos cada uno. ¿Qué orden tiene?
3. ¿Qué dos propiedades ayudan a garantizar la terminación?

SOLUCIONES EXPLICADAS

1. $O(n)$, porque el trabajo crece proporcionalmente a n .
2. $O(n^2)$, porque se realizan aproximadamente $n \times n$ pasos.
3. Una función estrictamente monótona y acotada.

TEMA 7

Funciones y procedimientos

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo dividimos un programa en miniprogramas reutilizables y decidimos qué datos entran, cuáles cambian y qué resultado vuelve?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Subprograma	Bloque con nombre que realiza una tarea concreta.
Función	Subprograma que devuelve un valor.
Procedimiento	Subprograma void: realiza una acción y no devuelve un valor con return.
Parámetro	Nombre usado en la definición del subprograma.
Argumento	Dato concreto enviado en una llamada.
Paso por valor	El subprograma recibe una copia.
Paso por referencia	El subprograma puede modificar la variable original.
Efecto lateral	Cambio observable fuera del valor devuelto.

Función frente a procedimiento

```
int cuadrado(int x) {           // función
    return x * x;
}

void incrementar(int &x) { // procedimiento
    x = x + 1;
}
```

La diferencia fundamental usada en el curso es que una función devuelve un valor. Un procedimiento puede modificar argumentos por referencia, escribir datos o realizar otras acciones, pero su tipo de retorno es void.

Valor, referencia y const

Forma	Qué recibe	¿Cambia el original?	Uso típico
int x	una copia	No	dato de entrada pequeño
int &x	la variable	Sí	resultado o dato que se modifica
const Tipo &x	la variable, solo lectura	No	entrada grande sin copiar
const Tipo x[]	vector abierto, solo lectura	No	recorrer un array recibido

Ejemplo de examen: sigue la llamada

```
void cambiar(int &a, int b) {
    a = a + 2;
    b = b + 5;
}

int x = 3;
int y = 4;
cambiar(x, y);
```

Momento	x	y	a	b
Antes de llamar	3	4	-	-
Al entrar	3	4	referencia a x	copia 4
a = a + 2	5	4	5	4
b = b + 5	5	4	5	9
Al volver	5	4	desaparece	desaparece

ERROR ROJO: ALIASING

Si pasas la misma variable a dos parámetros por referencia, ambos nombres apuntan al mismo dato. Una modificación mediante uno afecta al otro. Esta doble referencia puede hacer que un razonamiento normal falle.

Función pura

Una función pura no produce efectos laterales y calcula su resultado únicamente a partir de sus argumentos. En el enfoque del curso, una función con todos sus argumentos por valor y sin modificar estado exterior es el ejemplo típico de pureza.

Recursión, con una escalera

Una función recursiva se llama a sí misma con un problema más pequeño. Necesita un **caso base** que detenga las llamadas y un **paso recursivo** que se acerque a él.

```
int factorial(int n) {
    if (n == 0) return 1; // caso base
    return n * factorial(n - 1); // problema más pequeño
}
```

Para factorial(4), las llamadas bajan 4, 3, 2, 1, 0. Al llegar a 0 se devuelve 1. Después se reconstruye: 1 x 1 = 1; 2 x 1 = 2; 3 x 2 = 6; 4 x 6 = 24. No se multiplican todos de golpe: primero se baja y después se vuelve.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. Tras cambiar(x,y) en el ejemplo, ¿cuánto valen x e y?
2. ¿Qué falta en una recursión que nunca deja de llamarse?
3. ¿Qué significa const en un parámetro de entrada?

SOLUCIONES EXPLICADAS

1. x vale 5 porque se pasó por referencia; y sigue valiendo 4 porque b era una copia.
2. Un caso base alcanzable, o un paso que se acerque a él.
3. Que el subprograma puede leer el dato, pero no modificarlo mediante ese parámetro.

TEMA 8

Metodología de desarrollo (III): dividir sin perder el control

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo convertimos un problema grande en piezas pequeñas que cooperan y pueden probarse por separado?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Descomposición	Dividir una tarea grande en subtareas.
Responsabilidad	Trabajo concreto que corresponde a un módulo o subprograma.
Cohesión	Grado en que una pieza se concentra en una sola tarea.
Acoplamiento	Dependencia entre piezas.
Contrato	Precondiciones, postcondiciones y efectos de una operación.
Prueba unitaria	Prueba de una pieza aislada.
Diagrama modular	Dibujo de los módulos y sus relaciones.

La regla de la cocina profesional

En lugar de que una sola función lea datos, valide, calcule, ordene e imprima, cada parte recibe una responsabilidad. Las piezas son más cortas y sus errores se localizan mejor.

Módulo	Responsabilidad	No debería hacer
leerDatos	Obtener y validar entradas.	Ordenar o imprimir informes.
calcularMedia	Calcular y devolver una media.	Pedir datos al usuario.
ordenar	Cambiar el orden de una colección.	Inventar elementos.
mostrarInforme	Presentar resultados.	Modificar los cálculos.

MEMORIA

Buena cohesión: cada pieza tiene una misión clara. Bajo acoplamiento: una pieza conoce lo mínimo necesario de las demás.

Refinamiento sucesivo

Partimos de una frase general y la sustituimos por otras más concretas hasta que cada acción se puede programar directamente.

Nivel	Descripción
0	Gestionar calificaciones.
1	Leer notas; calcular media; decidir estado; mostrar resultado.
2	Validar cada nota; sumar; dividir; comparar con 5; imprimir mensaje.
3	Escribir las sentencias y subprogramas concretos.

Diseño descendente y pruebas

Puedes escribir primero las cabeceras de los subprogramas y luego completar cada cuerpo. Prueba las piezas simples antes de unir las. Si falla el programa completo, podrás saber qué contrato se incumple.

```
bool notaValida(float nota);  
float calcularMedia(const float notas[], int n);  
void mostrarResultado(float media);
```

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. ¿Es buena idea que calcularMedia también pida los datos por teclado?
2. ¿Qué refleja un diagrama de estructura modular?
3. ¿Qué combinación buscamos: cohesión alta o baja; acoplamiento alto o bajo?

SOLUCIONES EXPLICADAS

1. Normalmente no. Mezclar cálculo y entrada reduce la cohesión y dificulta las pruebas.
2. Los módulos y las relaciones entre ellos.
3. Cohesión alta y acoplamiento bajo.

TEMA 9

Definición de tipos

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo damos nombres y significado propios a los datos para que el programa se parezca al problema real?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Tipo predefinido	Tipo incluido en el lenguaje: int, float, char, bool.
Tipo sinónimo	Nuevo nombre para un tipo ya existente.
typedef	Palabra usada para declarar un nombre de tipo.
Enumerado	Tipo cuyos posibles valores se enumeran con nombres.
Compatibilidad	Regla que decide si dos valores pueden asignarse o compararse.
Conversión explícita	Conversión escrita int(...), float(...) u otra forma equivalente.

Un tipo añade lenguaje al problema

```
typedef int Identificador;  
typedef enum Dia { Lunes, Martes, Miercoles, Jueves, Viernes };  
  
Identificador cliente = 25;  
Dia hoy = Miercoles;
```

Identificador es un tipo sinónimo: renombra un tipo existente. Dia es un enumerado: limita y nombra los valores posibles. El código se vuelve más legible que si usáramos números sin explicar.

Qué valores reciben los enumerados

Si no se especifican números, el primer valor suele corresponder a 0 y los siguientes aumentan de uno en uno. En el ejemplo: Lunes=0, Martes=1 y Miercoles=2. No conviene depender de ello sin revisar la declaración.

Declaración	Qué es Tipo1	Qué es Tipo2
Tipo1 Tipo2;	un tipo	una variable de Tipo1
typedef Tipo1 Tipo2;	un tipo ya definido	un nuevo nombre de tipo
Tipo1 Tipo2 = {1,2,3};	un tipo	una variable inicializada

ERROR ROJO

No decidas si un nombre es tipo o variable por su aspecto. Lee la gramática de la declaración de izquierda a derecha y localiza typedef si existe.

Ejemplo: estados de un pedido

```
typedef enum Estado { Pendiente, Preparando, Enviado, Entregado };

Estado e = Preparando;
if (e == Preparando) {
    printf("Todavía no ha salido");
}
```

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. En `typedef int Edad;` ¿Edad es variable o tipo?
2. Si `enum Nivel { Bajo, Medio, Alto }`, ¿qué valor entero suele tener Alto?
3. ¿Por qué un enumerado es mejor que usar 0, 1 y 2 sin nombres?

SOLUCIONES EXPLICADAS

1. Es un nuevo nombre de tipo, sinónimo de `int`.
2. 2, porque el primero suele empezar en 0.
3. Limita los valores y hace visible su significado, reduciendo errores.

TEMA 10

Ampliación de estructuras de control

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo expresamos selecciones múltiples, bucles anidados y situaciones excepcionales sin crear un laberinto?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
switch	Selección entre varios casos según una expresión.
case	Etiqueta constante de una alternativa del switch.
break	Finaliza el caso o bucle más cercano.
default	Alternativa usada si ningún case coincide.
Bucle anidado	Bucle situado dentro de otro.
Excepción	Situación anómala que altera el flujo normal.
throw	Lanza una excepción cuyo dato es una expresión.

switch: una expresión elige un caso

```
switch (opcion) {  
    case 1: printf("Alta"); break;  
    case 2: printf("Consulta"); break;  
    default: printf("Opcion no valida");  
}
```

opcion es una expresión. Cada case lleva un valor constante y después aparecen sentencias. Sin break, la ejecución puede continuar por el caso siguiente.

Bucles anidados: reinicia el interior

```
int fila = 1;  
while (fila <= 3) {  
    int columna = 1;  
    while (columna <= 4) {  
        printf("%d,%d", fila, columna);  
        columna++;  
    }  
    fila++;  
}
```

Se imprimen 12 pares. El contador interior debe inicializarse dentro del bucle exterior para volver a empezar en cada fila. Este detalle aparece mucho al transformar un for en while.

Excepciones sin misterio

Una excepción separa el camino normal del tratamiento de un error. En throw Algo;, Algo es una expresión cuyo valor se lanza. Un bloque catch compatible puede recibirlo.

```
try {  
    if (divisor == 0) throw 0;  
    resultado = dividendo / divisor;  
} catch (int codigo) {  
    printf("Division imposible");  
}
```

DECISIÓN

Usa if para alternativas ligadas a condiciones generales; switch para varios valores discretos de una misma expresión; bucles para repetir; excepciones para separar un fallo del camino normal.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. En case Uno: Dos; break; ¿qué es Dos?
2. ¿Qué ocurre si el bucle interior no reinicia su contador?
3. En throw x + 1; ¿qué se lanza?

SOLUCIONES EXPLICADAS

1. Dos representa una sentencia o secuencia de sentencias del caso; Uno debe ser una etiqueta constante válida.
2. Después de la primera vuelta exterior puede no ejecutarse de nuevo, porque su contador ya quedó fuera del límite.
3. El valor de la expresión x + 1.

TEMA 11

Estructuras de datos: arrays, cadenas y registros

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo agrupamos muchos datos relacionados sin crear una variable distinta para cada uno?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Dato estructurado	Dato formado por varios componentes.
Array o formación	Colección de elementos del mismo tipo e indexados.
Índice	Posición usada para acceder a un elemento.
Cadena	Array de caracteres terminado según la convención de C.
Matriz	Array con dos dimensiones, como una tabla.
Registro o struct	Agrupar campos que pueden tener tipos diferentes.
union	Campos alternativos que comparten almacenamiento.

Array: un archivador numerado

```
int notas[5] = {7, 5, 9, 4, 8};  
printf("%d", notas[2]); // imprime 9
```

Los índices comienzan en 0. En un array de 5 elementos, los índices válidos son 0, 1, 2, 3 y 4. El quinto elemento es `notas[4]`.

Índice	0	1	2	3	4
Valor	7	5	9	4	8

ERROR ROJO

Acceder a `notas[5]` está fuera del array. No es el quinto elemento: es una posición posterior al final.

Registro: una ficha con campos diferentes

```
typedef struct Vivienda {  
    char direccion[60];  
    int habitaciones;  
    float precio;  
    bool disponible;  
};  
  
Vivienda v;  
v.habitaciones = 3;
```

Un array responde: 'tengo muchas cosas del mismo tipo'. Un registro responde: 'una cosa tiene varias características'. Un catálogo suele ser un array de registros: muchas viviendas, cada una con sus campos.

Cadenas

Una cadena es una formación de caracteres con una marca final nula. Por eso, una cadena de hasta 20 caracteres necesita normalmente espacio adicional para el terminador. No la trates como un número ni asumas que todos los operadores de otros lenguajes funcionan igual.

Registro con variantes

Cuando un dato puede tener formas alternativas, se combinan un campo que indica la variante y una union con los campos alternativos. `struct` organiza la ficha y `union` permite compartir el espacio de las variantes.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. ¿Cuál es el último índice válido de `int datos[10]`?
2. ¿Pueden los elementos de un array tener tipos diferentes?
3. ¿Qué usarías para representar una persona con nombre, edad y altura?

SOLUCIONES EXPLICADAS

1. 9, porque los índices válidos van de 0 a 9.
2. No. Todos tienen el mismo tipo, aunque ese tipo puede ser simple o estructurado.
3. Un registro, porque agrupa campos relacionados de tipos distintos.

TEMA 12

Operaciones típicas con formaciones

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo recorreremos, busquemos y ordenemos un array sin perdernos con los índices?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Recorrido	Visitar sistemáticamente todos los elementos.
Acumulación	Construir un resultado combinando elementos.
Búsqueda lineal	Examinar desde el principio hasta encontrar o terminar.
Búsqueda binaria	Descartar la mitad de un array ya ordenado.
Ordenación	Reorganizar según un criterio.
Intercambio	Cambiar de posición dos elementos usando un auxiliar.
Centinela	Valor o condición que indica el final o un hallazgo.

Recorrido: el patrón que debes reconocer

```
int suma = 0;
for (int i = 0; i < n; i++) {
    suma = suma + datos[i];
}
```

La inicialización ocurre antes del bucle. La condición usa $i < n$, no $i \leq n$. Cada vuelta procesa exactamente `datos[i]`.

Búsqueda lineal

```
int posicion = -1;
int i = 0;
while (i < n && posicion == -1) {
    if (datos[i] == buscado) posicion = i;
    i++;
}
```

-1 significa 'no encontrado'. El bucle termina al agotar el array o al hallar el dato. En el peor caso revisa n posiciones: $O(n)$.

Búsqueda binaria: solo si está ordenado

Compara con el elemento central. Si el buscado es menor, continúa en la mitad izquierda; si es mayor, en la derecha. Cada paso reduce a la mitad el espacio: $O(\log n)$. Aplicarla a datos desordenados da resultados incorrectos.

Ordenación por selección, paso a paso

Ejemplo: [4, 2, 5, 1]. En cada posición buscamos el menor de la parte que falta y lo intercambiamos.

Paso	Parte ya ordenada	Parte pendiente	Acción
Inicio	[]	[4,2,5,1]	El menor es 1.

Paso	Parte ya ordenada	Parte pendiente	Acción
1	[1]	[2,5,4]	Intercambiamos 4 y 1.
2	[1,2]	[5,4]	El menor pendiente es 2; ya está.
3	[1,2,4]	[5]	Intercambiamos 5 y 4.
Fin	[1,2,4,5]	[]	Orden terminado.

La selección utiliza bucles anidados y su coste típico es $O(n^2)$. La ordenación debe comparar el campo correcto cuando los elementos son registros, por ejemplo `viviendas[i].precio`.

Intercambio seguro

```
Tipo aux = datos[i];
datos[i] = datos[j];
datos[j] = aux;
```

Sin la variable auxiliar perderíamos el valor original de `datos[i]` al sobrescribirlo.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. ¿Por qué una búsqueda binaria no sirve en un array desordenado?
2. ¿Qué orden tiene recorrer una vez n elementos?
3. Ordena [3,1,2] por selección: ¿qué queda tras colocar el primer mínimo?

SOLUCIONES EXPLICADAS

1. Porque decidir qué mitad descartar solo es correcto si los datos están ordenados.
2. $O(n)$.
3. [1,3,2]: el 1 se intercambia con el primer elemento.

TEMA 13

Punteros y variables dinámicas

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo guardamos la dirección de un dato y creamos información cuyo tamaño o existencia se decide durante la ejecución?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Dirección	Lugar de memoria donde vive un dato.
Puntero	Variable que guarda una dirección.
Desreferenciar	Acceder al dato situado en la dirección.
new	Reserva memoria para un dato dinámico y devuelve un puntero.
delete	Libera memoria previamente reservada.
NULL/null	Valor que indica que el puntero no señala un objeto válido.
Fuga de memoria	Memoria reservada que ya no puede liberarse.
Puntero colgante	Puntero que conserva la dirección de memoria ya liberada.

La imagen del papel con una dirección

Una variable normal es una casa con un contenido. Un puntero es un papel donde está escrita la dirección de una casa. El papel no es la casa ni el contenido: solo permite llegar a él.

```
int *p = new int;
*p = 25;
printf("%d", *p); // imprime 25
delete p;
p = NULL;
```

Expresión	Significado
int *p	p es un puntero capaz de señalar un int.
new int	crea un int dinámico y produce su dirección.
*p = 25	guarda 25 en el int señalado.
delete p	libera ese int dinámico.
p = NULL	deja explícito que ya no señala un objeto válido.

PREGUNTA TÍPICA

En `Uno = new Dos;`, Uno debe ser un puntero y Dos es el tipo de dato que se reserva.

Estructura enlazada mínima

```
typedef struct Nodo {
    int dato;
    Nodo *siguiente;
};

Nodo *primero = new Nodo;
primero->dato = 7;
primero->siguiente = NULL;
```

Cada nodo guarda un dato y la dirección del siguiente. A diferencia de un array fijo, la cadena puede crecer nodo a nodo. El operador -> accede a un campo a través de un puntero.

Tres errores que debes visualizar

Error	Qué ocurre	Prevención
Desreferenciar NULL	Intentamos entrar en una casa inexistente.	Comprobar p != NULL.
Fuga	Perdemos la dirección antes de delete.	Liberar cada reserva cuando deja de usarse.
Colgante	Usamos una dirección después de delete.	No usarla y asignar NULL.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. En p = new Tipo; ¿qué es p y qué es Tipo?
2. ¿Qué hace *p si p señala un entero válido?
3. ¿Por qué conviene poner p = NULL tras delete p?

SOLUCIONES EXPLICADAS

1. p es un puntero; Tipo es el tipo de dato que se crea dinámicamente.
2. Accede al entero señalado por p.
3. Evita conservar una dirección ya inválida y permite detectar que no señala nada.

TEMA 14

Tipos Abstractos de Datos (TAD)

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo diseñamos un tipo que proteja sus datos y ofrezca únicamente operaciones con significado?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Abstracción	Quedarse con lo esencial y apartar detalles internos.
TAD	Tipo definido por sus valores posibles y sus operaciones observables.
Interfaz	Qué operaciones puede usar el resto del programa.
Implementación	Cómo se representan los datos y se programan las operaciones.
Encapsulación	Agrupar datos y operaciones relacionadas.
Ocultación	Impedir que el exterior dependa de detalles internos.
Invariante de representación	Propiedad que los datos internos deben mantener siempre.

El mando a distancia

Puedes usar un televisor con encender, apagar y cambiar Canal sin conocer sus circuitos. La interfaz es el mando; la implementación son los circuitos. En un TAD, el programa cliente conoce las operaciones, pero no debería tocar directamente la representación interna.

PROPÓSITO DE LA OCULTACIÓN

Permitir cambios en la implementación interna sin afectar al código que usa el TAD. No busca esconder nuestra identidad ni acelera automáticamente el programa.

Ejemplo: TAD Cuenta

Parte	Diseño
Estado interno	titular y saldo
Invariante	saldo $\geq$ 0
Operaciones	Crear, Ingresar, Retirar, ConsultarSaldo
Regla	Retirar falla si el importe supera el saldo.

```
// Interfaz conceptual
Cuenta Crear(const char nombre[]);
void Ingresar(Cuenta &c, float cantidad);
bool Retirar(Cuenta &c, float cantidad);
float ConsultarSaldo(const Cuenta &c);
```

La interfaz explica lo que se puede pedir. La implementación podría guardar céntimos enteros en vez de euros reales sin obligar a cambiar el programa cliente, siempre que mantenga el contrato.

Cómo convertir un enunciado en un TAD

Paso	Pregunta	Resultado
1	¿Qué cosa se gestiona?	Nombre del TAD.
2	¿Qué datos describe cada elemento?	Registro de elemento.
3	¿Cuántos elementos caben?	Capacidad fija y contador.
4	¿Qué acciones exige el enunciado?	Cabeceras de operaciones.
5	¿Qué siempre debe ser cierto?	Invariantes y validaciones.
6	¿Qué ocurre si no se puede?	Valor de retorno y casos límite.

Registro no es igual a TAD

Un registro solo agrupa campos. Un TAD incluye además una especificación de operaciones y una barrera que protege la representación. En los ejercicios suele usarse un registro dentro de la implementación del TAD, pero no son sinónimos.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. ¿Qué debe conocer el programa cliente: interfaz o representación interna?
2. ¿Cuál es la diferencia entre registro y TAD?
3. En una colección de máximo 100 elementos, ¿qué dato interno suele acompañar al array?

SOLUCIONES EXPLICADAS

1. La interfaz: operaciones y contratos. La representación interna debe poder cambiar.
2. El registro agrupa campos; el TAD define valores, operaciones y ocultación.
3. Un contador de cuántas posiciones están realmente ocupadas.

TEMA 15

Módulos y resolución del ejercicio largo

LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo separamos interfaz e implementación y cómo construimos, paso a paso, una solución de tamaño examen?

Palabras nuevas, explicadas antes de usarlas

Palabra	Significado sencillo
Módulo	Unidad que agrupa declaraciones y operaciones relacionadas.
Módulo de definición	Publica la interfaz.
Módulo de implementación	Contiene la representación y el código.
Cliente	Código que usa el módulo.
Cabecera	Nombre, parámetros y resultado de un subprograma.
Dependencia	Relación por la que una pieza necesita otra.
Trazabilidad	Correspondencia entre requisitos, operaciones y pruebas.

Dos habitaciones separadas

El módulo de definición es el mostrador: enseña lo que se puede pedir. El módulo de implementación es el taller: contiene los datos privados y el modo de realizar cada operación. El cliente no entra en el taller.

```
// Definición pública, idea simplificada
TAD Biblioteca;
bool AgregarLibro(Biblioteca &b, const Libro &nuevo);
bool Prestar(Biblioteca &b, int id);
void ListarDisponibles(const Biblioteca &b);

// Implementación privada, idea simplificada
typedef struct Biblioteca {
    Libro libros[50];
    int usados;
};
```

En una notación del estilo `void Uno::Dos()`, se está escribiendo la implementación de la operación `Dos` asociada al tipo o ámbito `Uno`.

Ejercicio completo guiado: biblioteca

ENUNCIADO RESUMIDO

Diseña un TAD para una biblioteca de hasta 50 libros. Cada libro tiene identificador, título y estado de préstamo. Debe permitir agregar un libro, prestar por identificador, devolver y listar disponibles ordenados por título.

Paso 1. Extraer sustantivos y límites

Sustantivos: biblioteca, libro, identificador, título, estado. Límite: 50 libros. Esto sugiere un registro `Libro`, un array de 50 y un contador de usados.

```
const int MAX_LIBROS = 50;

typedef struct Libro {
    int id;
    char titulo[60];
    bool prestado;
};

typedef struct Biblioteca {
    Libro libros[MAX_LIBROS];
    int usados;
};
```

Paso 2. Escribir el invariante

Debe cumplirse $0 \leq \text{usados} \leq \text{MAX_LIBROS}$. Las posiciones válidas están entre 0 y usados-1. No debe haber dos libros con el mismo identificador.

Paso 3. Inicializar

```
void Inicializar(Biblioteca &b) {
    b.usados = 0;
}
```

Paso 4. Crear una búsqueda auxiliar

```
int BuscarPorId(const Biblioteca &b, int id) {
    int i = 0;
    while (i < b.usados && b.libros[i].id != id) {
        i++;
    }
    if (i < b.usados) return i;
    return -1;
}
```

La función devuelve la posición o -1. Al reutilizarla en Prestar y Devolver evitamos duplicar el mismo recorrido.

Paso 5. Agregar con todas las decisiones

```
bool AgregarLibro(Biblioteca &b, const Libro &nuevo) {
    if (b.usados == MAX_LIBROS) return false;
    if (BuscarPorId(b, nuevo.id) != -1) return false;

    b.libros[b.usados] = nuevo;
    b.libros[b.usados].prestado = false;
    b.usados++;
    return true;
}
```

Primero protegemos los límites. Solo después escribimos en el array. La nueva posición es usados; tras ocuparla, incrementamos el contador.

Paso 6. Prestar

```
bool Prestar(Biblioteca &b, int id) {
    int pos = BuscarPorId(b, id);
    if (pos == -1) return false;
    if (b.libros[pos].prestado) return false;
    b.libros[pos].prestado = true;
    return true;
}
```

Paso 7. Ordenar sin romper la colección

La operación de listado puede ordenar una copia si no queremos modificar el orden interno. Si el enunciado exige que la colección quede ordenada, ordenamos el array real. Debemos intercambiar registros Libro completos, no solo los títulos.

Paso 8. Pruebas mínimas

Prueba	Resultado esperado
Biblioteca vacía	Listar no imprime libros y no accede a la posición 0.
Agregar primer libro	usados pasa de 0 a 1; queda no prestado.
Identificador repetido	Agregar devuelve false y usados no cambia.
Prestar inexistente	Devuelve false sin modificar datos.
Prestar dos veces	La segunda llamada devuelve false.
Capacidad completa	El libro 51 no se escribe fuera del array.

LO QUE PUNTÚA

Una solución clara declara los tipos, mantiene el contador, respeta los límites, escribe cabeceras coherentes, reutiliza auxiliares, contempla fallos y explica las decisiones. No empieces por el cuerpo de las operaciones sin diseñar antes la representación.

Comprueba si lo has entendido

HAZLO SIN MIRAR LA SOLUCIÓN

1. ¿Qué publica el módulo de definición?
2. ¿Por qué BuscarPorId mejora el diseño?
3. Al ordenar libros, ¿se intercambia solo el título o el registro entero?

SOLUCIONES EXPLICADAS

1. La interfaz: tipos visibles, cabeceras y contratos necesarios para usar el módulo.
2. Centraliza una responsabilidad, evita código duplicado y permite probar la búsqueda por separado.
3. El registro entero, para que id, título y estado sigan perteneciendo al mismo libro.

Método para resolver el examen

FORMATO VERIFICADO EN LA GUÍA PÚBLICA 2026/27

120 minutos, sin material, 10 preguntas tipo test y un ejercicio. El test es eliminatorio. Antes del examen confirma en el curso virtual que no haya cambios y revisa el algoritmo final de calificación.

Reparto orientativo del tiempo

Tiempo	Acción
0-5 min	Lee todo. Marca las preguntas de certeza alta.
5-30 min	Resuelve el test con trazas breves al lado.
30-35 min	Revisa que hay una sola opción por pregunta.
35-50 min	Diseña el ejercicio: tipos, representación, invariante y cabeceras.
50-105 min	Implementa operaciones de una en una.
105-120 min	Comprueba límites, referencias, índices, retornos y legibilidad.

Algoritmo para una pregunta de seguimiento

1. Copia los valores iniciales en una tabla.
2. Decide el tipo de cada operación: división entera, resto, comparación o conversión.
3. En una llamada, marca V para copia y R para referencia.
4. Ejecuta una sola sentencia y actualiza la tabla.
5. En un bucle, vuelve a evaluar la condición con los valores nuevos.
6. Compara tu resultado con las opciones solo al final.

NO HAGAS ESTO

No intentes ejecutar todo mentalmente de un salto. Las opciones incorrectas están diseñadas para coincidir con errores frecuentes: olvidar una actualización, confundir / con %, tratar una referencia como copia o saltarse una vuelta.

Plantilla del ejercicio de TAD

1. Constantes de capacidad
2. Enumerados necesarios
3. Registro de un elemento
4. Representación del TAD: array + contador + otros datos
5. Invariante de representación
6. Cabeceras de todas las operaciones
7. Funciones auxiliares: buscar, ordenar, validar
8. Implementación operación por operación
9. Casos límite y valores de fallo

Hoja de memoria rápida

NO LA MEMORICES EL PRIMER DÍA

Úsala al final de cada sesión para recuperar las ideas sin releer el capítulo.

Símbolos que no puedes confundir

Símbolo	Significado	Pregunta
=	asignar	¿Qué valor guardo?
==	comparar igualdad	¿Son iguales?
%	resto entero	¿Qué sobra al dividir?
& en parámetro	referencia	¿Puede cambiar el original?
*p	dato señalado	¿Qué hay en la dirección?
p->campo	campo a través de puntero	¿Qué campo del objeto señalado?
[]	índice o parte opcional según contexto	¿Array o notación BNF?
{ }	bloque o repetición BNF según contexto	¿Código o metanotación?

Patrones de código

```
// Recorrido
for (int i = 0; i < n; i++) { usar(v[i]); }

// Búsqueda
int i = 0;
while (i < n && v[i] != buscado) { i++; }
bool encontrado = (i < n);

// Contar
int cuenta = 0;
for (int i = 0; i < n; i++) {
    if (cumple(v[i])) cuenta++;
}

// Acumular
float total = 0;
for (int i = 0; i < n; i++) { total += v[i]; }
```

Preguntas de control del TAD

- ¿Está declarada la capacidad máxima?
- ¿El contador empieza en cero y nunca supera la capacidad?
- ¿Cada acceso al array usa un índice válido?
- ¿Cada operación indica claramente entrada, modificación y resultado?
- ¿Se contempla lleno, vacío, inexistente y repetido?
- ¿Al ordenar se mueve el registro completo?
- ¿La interfaz evita que el cliente toque los datos internos?

Simulacro de consolidación

Haz primero las 20 preguntas sin mirar la corrección. No se resta por error en el formato histórico revisado, pero en el examen oficial debes seguir las instrucciones impresas ese día.

1. ¿Qué describe BNF? A) la sintaxis B) la velocidad C) la semántica D) la memoria
2. Si `int x=7; int y=2;` ¿qué vale `x/y`? A) 2 B) 3 C) 3.5 D) 4
3. ¿Cuál es el último índice válido de `int v[8]`? A) 8 B) 7 C) 6 D) depende
4. En `void f(int &x)`, `x` se pasa: A) por valor B) por referencia C) como constante D) como tipo
5. ¿Qué estructura ejecuta el cuerpo al menos una vez? A) `if` B) `while` C) `do-while` D) `switch`
6. Un recorrido completo de `n` elementos suele ser: A) $O(1)$ B) $O(\log n)$ C) $O(n)$ D) $O(n^2)$
7. En `p = new Tipo`, `p` debe ser: A) registro B) puntero C) constante D) módulo
8. Una función pura: A) cambia globales B) no tiene efectos laterales C) siempre es `void` D) siempre recibe referencias
9. El propósito principal de ocultar un TAD es: A) usar menos tinta B) permitir cambiar la implementación C) eliminar pruebas D) evitar tipos
10. En `switch(x)`, `x` es: A) una expresión B) un procedimiento C) un módulo D) necesariamente un puntero
11. Un registro: A) solo contiene enteros B) agrupa campos C) siempre es dinámico D) es igual a un TAD
12. Para búsqueda binaria, el array debe estar: A) lleno B) ordenado C) vacío D) duplicado
13. Si `x` se pasa por valor: A) se modifica el original B) se crea una copia C) es un puntero D) no tiene tipo
14. `typedef int Edad` declara: A) una variable B) un procedimiento C) un tipo sinónimo D) un array
15. Para garantizar terminación buscamos una función: A) aleatoria B) monótona y acotada C) constante siempre D) sin límite
16. Dos bucles completos anidados suelen costar: A) $O(\log n)$ B) $O(n)$ C) $O(n^2)$ D) $O(1)$
17. `throw Algo;` lanza: A) el compilador B) una expresión C) un módulo D) un tipo sin valor
18. Un array contiene elementos: A) del mismo tipo B) siempre distintos C) sin índices D) solo enteros
19. Al insertar en un array de capacidad 100 y usados=100: A) se usa posición 100 B) se amplía solo C) debe rechazarse D) se borra el primero
20. Una función recursiva necesita: A) solo globales B) un caso base alcanzable C) dos arrays D) `switch`

CORRECCIÓN RAZONADA

- 1 **A.** BNF especifica formas sintácticas válidas.
- 2 **B.** La división entre enteros descarta decimales: $7/2=3$.
- 3 **B.** Ocho posiciones van de 0 a 7.
- 4 **B.** & indica paso por referencia.
- 5 **C.** do-while comprueba después del cuerpo.
- 6 **C.** Se visita cada elemento una vez.
- 7 **B.** new devuelve una dirección.
- 8 **B.** La pureza evita efectos laterales.
- 9 **B.** El cliente depende de la interfaz, no de la representación.
- 10 **A.** switch evalúa una expresión.
- 11 **B.** Un registro agrupa campos relacionados.
- 12 **B.** Solo el orden permite descartar mitades.
- 13 **B.** El parámetro recibe una copia.
- 14 **C.** Edad es otro nombre para int.
- 15 **B.** Debe avanzar estrictamente y no poder hacerlo para siempre.
- 16 **C.** n por n produce n^2 pasos.
- 17 **B.** Se lanza el valor de Algo.
- 18 **A.** Todos los elementos comparten tipo.
- 19 **C.** La colección está llena; escribir sería salir del array.
- 20 **B.** Sin caso base alcanzable no se detienen las llamadas.

Taller de trazas

Traza 1

```
int x = 10;
int y = 3;
x = x % y;
y = y + x * 2;
```

Antes de ver la solución, dibuja dos columnas: x e y.

SOLUCIÓN

10 % 3 es 1, por lo que x pasa a 1. Después $y = 3 + 1 \times 2 = 5$. Resultado: x=1, y=5.

Traza 2

```
int total = 0;
for (int i = 1; i <= 5; i++) {
    if (i % 2 == 0) total += i;
}
```

SOLUCIÓN

Solo se suman los pares 2 y 4. total evoluciona 0, 2, 6. Resultado final: 6.

Traza 3

```
void doble(int &a, int b) {
    a = a * 2;
    b = b * 2;
}
int x = 3;
doble(x, x);
```

SOLUCIÓN

a es referencia al x original; b recibe una copia del valor inicial 3. a cambia x a 6. Después b pasa a 6, pero esa copia desaparece. x termina valiendo 6.

Traza 4

```
int v[4] = {3, 1, 4, 2};
int mayor = v[0];
for (int i = 1; i < 4; i++) {
    if (v[i] > mayor) mayor = v[i];
}
```

SOLUCIÓN

mayor empieza en 3. Se compara 1 y no cambia; 4 es mayor y lo sustituye; 2 no cambia. Resultado: 4.

Práctica de ejercicios largos

No intentes programarlos enteros de una vez. Para cada uno entrega primero representación, invariante y cabeceras. Después implementa una operación por sesión.

Problema A: agenda veterinaria

ENUNCIADO

Gestiona hasta 80 citas. Cada cita tiene identificador, nombre del animal, día, hora y estado. Operaciones: nueva cita si no está llena ni existe el identificador; cancelar; listar las citas activas de un día ordenadas por hora.

Pistas: registro Cita; array de 80; contador; enum Estado; BuscarPorId; filtro por día; ordenación por hora. Prueba vacío, lleno, repetido e inexistente.

Problema B: almacén de componentes

ENUNCIADO

Gestiona hasta 120 componentes. Cada uno tiene código, descripción, unidades y precio. Operaciones: alta, entrada de unidades, salida si hay stock suficiente, valor total e informe ordenado por código.

Pistas: el valor de una línea es unidades x precio. La salida debe validar existencia y stock. El valor total es una acumulación. Ordena registros completos.

Problema C: cursos de una academia

ENUNCIADO

Gestiona 30 cursos. Cada curso tiene código, nombre, plazas, inscritos y modalidad. Operaciones: crear curso, matricular si quedan plazas, anular matrícula y listar por modalidad ordenado por plazas libres.

Pistas: enum Modalidad; invariante $0 \leq \text{inscritos} \leq \text{plazas}$; plazas libres = plazas - inscritos. Decide con claridad qué devuelve cada operación cuando falla.

Rúbrica para autocorregir cada problema

Criterio	Sí/No
He identificado todos los datos y límites.	
Mi representación incluye array y contador coherentes.	
He escrito un invariante.	
Cada cabecera distingue entrada, referencia y resultado.	
Compruebo lleno, vacío, repetido e inexistente cuando corresponde.	
Ningún índice puede salirse de $0..usados-1$.	
Reutilizo una búsqueda auxiliar.	
La ordenación mueve registros completos.	
Puedo explicar cada línea sin leerla.	

Plan de dominio en 30 sesiones

Está pensado para una hora por sesión de esta asignatura. Puedes incluirlo dentro de tus tres horas diarias sin que Fundamentos de Programación absorba todo el tiempo destinado a las demás materias.

Sesiones	Contenido	Meta
1-2	Temas 1-2	Lenguaje, BNF, expresiones y operadores
3-4	Tema 3	Variables, tipos, conversiones y ámbito
5-6	Temas 4-5	Especificación, if, while, do-while y for
7-8	Tema 6	Corrección, terminación y complejidad
9-11	Tema 7	Funciones, valor, referencia, pureza y recursión
12-13	Temas 8-9	Descomposición, módulos iniciales y tipos propios
14-15	Tema 10	switch, bucles anidados y excepciones
16-18	Tema 11	Arrays, cadenas, registros y variantes
19-21	Tema 12	Recorridos, búsqueda y ordenación
22-23	Tema 13	Punteros y memoria dinámica
24-26	Temas 14-15	TAD, interfaz, implementación y módulos
27	Test	20 preguntas y corrección explicada
28	Trazas	Cuatro trazas y dos nuevas de exámenes
29	TAD	Diseñar un ejercicio completo sin programar
30	Simulacro	120 minutos con examen histórico

CÓMO SABER SI PUEDES AVANZAR

No exijas perfección. Avanza cuando puedas explicar la idea sin leer, acertar al menos 4 de 5 microejercicios y seguir una traza sin saltarte pasos. Vuelve a los errores en 24 horas y una semana después.

Glosario esencial

Término	Definición corta
Acoplamiento	Dependencia entre módulos.
Algoritmo	Pasos finitos y precisos para resolver un problema.
Aliasing	Dos nombres o referencias que acceden al mismo dato.
Array	Colección indexada de elementos del mismo tipo.
Ámbito	Zona donde un nombre es visible.
BNF	Notación para describir sintaxis.
Bucle	Repetición controlada de instrucciones.
Compilador	Traduce código fuente y comprueba su forma.
Complejidad	Crecimiento de recursos con el tamaño de entrada.
Encapsulación	Agrupar datos y operaciones.
Función	Subprograma que devuelve un valor.
Invariante	Propiedad que se conserva.
Módulo	Unidad de organización con una responsabilidad.
Parámetro	Nombre formal que recibe datos en un subprograma.
Postcondición	Propiedad garantizada al terminar.
Precondición	Propiedad exigida antes de empezar.
Procedimiento	Subprograma void que realiza una acción.
Puntero	Variable que guarda una dirección.
Registro	Dato con campos posiblemente de tipos distintos.
Semántica	Significado de una construcción.
Sintaxis	Forma válida de escribir.
TAD	Tipo definido por valores y operaciones, con representación oculta.
Traza	Tabla de valores durante la ejecución.
Variable	Dato con nombre cuyo valor puede cambiar.

Fuentes, alcance y siguiente paso

Material utilizado

- Guía de estudio pública UNED 2026/27 de Fundamentos de Programación, código 71901020, especialmente contenidos y sistema de evaluación.
- Archivo personal organizado de 42 enunciados históricos entre 2010/11 y 2025/26.
- Archivo personal de 16 documentos de soluciones oficiales entre 2010/11 y 2019/20.
- Patrones recientes de examen observados entre 2023/24 y 2025/26 para orientar prioridades, sin convertirlos en una predicción.

Qué es este manual

Es un material original de apoyo, escrito desde cero para comprender el programa oficial y practicar los razonamientos que exige la asignatura. No reproduce el libro de texto ni sustituye las instrucciones del equipo docente, el curso virtual o el entorno C+/- de las prácticas.

Qué hacemos después

Cuando hayas leído los temas 1 y 2, haremos una prueba diagnóstica breve. Después trabajaremos en sesiones: tú respondes primero, yo corrijo el razonamiento, explico el error con una imagen mental y añado una regla de memoria. Más adelante resolveremos exámenes históricos completos sin mirar las soluciones.

PRIMERA META

Lee hasta el final del Tema 2. Debes poder explicar con tus palabras: algoritmo, programa, sintaxis, semántica, variable, expresión, sentencia, división entera y resto.

No necesitas entenderlo todo a la primera. La programación se aprende prediciendo, ejecutando, equivocándose y explicando por qué.